

Dokumentacja

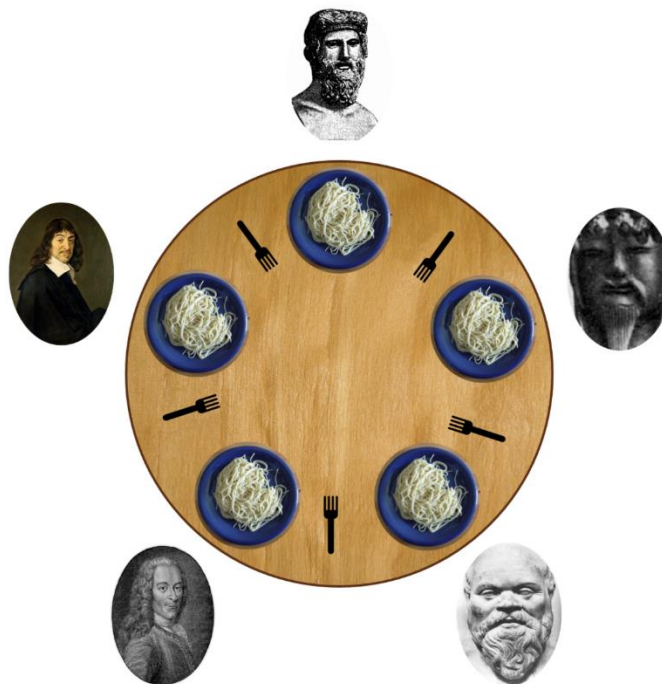
Projekt 1

Temat: Problem jedzących filozofów

Julia Sujka, 272869

1. Opis problemu

Problem jedzących filozofów polega na rozwiązaniu problemu synchronizacji wątków. Sytuacja wygląda następująco: Przy okrągłym stole siedzą filozofowie, na stole przed każdym z nich leży talerz z jedzeniem oraz dwa widelce prawy oraz lewy. Jest N filozofów i N widelców pomiędzy nimi. Filozofowie mogą być tylko w jednym z dwóch możliwych stanów, mogą albo jeść, albo myśleć. Czas danych stanów jest losowany. Aby jeść muszą podnieść oba widelce. Mogą podnieść tylko jeden widelec naraz. Jeśli podniesie jeden i będzie chciał podnieść drugi, który okaże się niedostępny odkłada pierwszy widelec. Powinno się unikać sytuacji, w której wszyscy filozofowie czekają na widelce i żaden nie może jeść (deadlock).



2. Instrukcja uruchomienia projektu

Program przyjmuje jako argument liczbę filozofów - musi być ona większa od 1. Po uruchomieniu należy również wprowadzić czas w sekundach przez który chcemy aby program się wykonywał.

3. Opis rozwiązania

Program symuluje problem jedzących filozofów w środowisku wielowątkowym. Liczba filozofów jest podawana jako argument programu, a czas działania jest wprowadzany przez użytkownika.

Każdy filozof działa w osobnym wątku i naprzemiennie myśli oraz je, używając dwóch widelców (mutexów) do jedzenia. Wątki działają równolegle, dzięki czemu mogą niezależnie próbować zdobyć zasoby (widelce). Po upływie określonego czasu program kończy działanie i podaje statystyki dotyczące liczby posiłków każdego filozofa.

`forks[N]` – dynamiczna lista mutexów reprezentujących widelce.

`score[N]` – przechowuje liczbę posiłków zjedzonych przez każdego filozofa.

`philosophers` – dynamiczna lista wątków.

```
for (int i = 0; i < N; i++) {  
    philosophers.emplace_back(philosopher, i, std::ref(forks), std::ref(score));  
}
```

Tworzy **N wątków**, uruchamiających funkcję `philosopher()`, przekazując im identyfikator, listę mutexów i wyników.

print_protect – mutex chroniący `std::cout`, aby uniknąć przeplatania się komunikatów z różnych wątków.

running – zmienna sterująca działaniem filozofów. Po upływie określonego czasu jest ustawiana na `false`, co powoduje zakończenie pracy wątków.

Funkcja raportująca stan filozofa

```
void report(int id, const std::string& state) {  
    std::lock_guard<std::mutex> lock(print_protect);  
    std::cout << "Philosopher " << id << " " << state << std::endl;  
}
```

Zabezpiecza dostęp do `std::cout` za pomocą `print_protect`.

Wyświetla aktualny stan filozofa (np. „thinking...”, „took the left fork”, „finished eating”).

Każdy filozof losowo myśli `sleep_for()` na losowy czas i raportuje swój stan. Próbuje podnieść lewy widelec (`lock()` – blokowanie mutexu). Próbuje podnieść prawy widelec jeśli się uda `try_lock()`, filozof je, zwiększa swój wynik i odkłada oba widelce. Jeśli się nie uda, odkłada lewy widelec i wraca do myślenia. Pętla działa, dopóki `running` jest `true`.

W funkcji main sprawdzamy, czy podano liczbę filozofów jako argument. Jeśli nie, wyświetla instrukcję i kończy działanie. Wczytujemy czas działania programu w sekundach.

Sekcja krytyczna to fragment kodu, w którym kilka wątków może próbować jednocześnie uzyskać dostęp do współdzielonego zasobu, co może prowadzić do błędów lub niepożądanego zachowania. W naszym programie sekcjami krytycznymi są:

- Dostęp do widelców (mutexy forks[])
- Dostęp do std::cout (mutex print_protect)

Aby uniknąć deadlocka filozof najpierw podnosi lewy widelec, a następnie próbuje podnieść prawy widelec przy pomocy try_lock(). Jeśli nie uda się podnieść prawego widelca, filozof odkłada lewy widelec i wraca do myślenia.

Każdy dostęp do std::cout jest **chroniony mutexem print_protect**, używając std::lock_guard. Dzięki temu tylko jeden filozof naraz może wypisywać komunikat, co zapobiega mieszaniu tekstu.